

해커톤 결과 보고서

다이어트 동아리 활동 기록 관리 웹앱 — 내 몸에 실제로 효과가 있었던 음식과 운동 찾기

팀번호 / 팀이름	1팀 / schoolLover
팀원 및 역할	이동제 — 프론트엔드 (<code>index.html</code> , <code>record.html</code> , <code>style.css</code> , <code>js/ui/</code>) 박효민 — 데이터 계층 (<code>js/store/store.js</code> , 설계서) 봉준표 — 데이터 처리 (<code>js/analytics/analytics.js</code> , <code>recommend.js</code>) 공통: 디버깅 · 유지보수
GitHub 저장소	<code>https://github.com/bongjunpyo/hackathon1schoolLover</code>
제출일	2026년 8월 5일

1. 문제 정의와 기획

이 웹앱이 해결하려는 문제

다이어트 동아리에서 활동 기록은 계속 쌓인다. 그런데 정작 중요한 질문에는 아무도 답하지 못한다.

지난 한 달 동안 내가 살이 빠진 건, **뭘** 해서 빠진 거지?

기존 기록 앱은 "무엇을 했는가"까지만 남긴다. 러닝 20회, 헬스 15회, 총 1,200분. 숫자는 많은데 **그중 어떤 게 실제로 체중을 움직였는지** 는 알 수 없다. 사람마다 반응하는 운동과 음식이 다른데도 남들이 좋다는 걸 그대로 따라 하게 되는 이유가 이것이다.

이 앱은 날짜별 체중 변화를 그날의 운동·음식에 귀속시켜 항목별 감량 효과 순위를 만든다. "닭가슴살을 먹은 날은 하루 평균 -0.115kg " 처럼 **내 데이터로 검증된 문장**을 만드는 것이 목표다.

주 사용자와 사용 상황

다이어트 동아리 임원과 회원. 운동을 마친 직후 휴대폰이나 PC에서 활동명·날짜·장소·참여 인원과 함께 당일 체중·섭취 칼로리를 한 줄로 남긴다. 한 달치가 쌓이면 통계 화면에서 추세를 확인한다.

구현한 필수 기능 3가지

기능	내용
활동 등록	활동명 · 날짜 · 장소 · 참여 인원 · 메모 입력 후 저장. 확장 필드(종목, 운동 시간, 체중, 섭취 칼로리, 운동 종류, 음식)를 함께 받는다
활동 목록 조회	최신순 정렬. 기록이 없으면 "아직 기록이 없습니다. 오늘 체중부터 남겨보세요." 안내 표시
활동 삭제	항목별 삭제 버튼. <code>confirm()</code> 확인 절차를 거친 뒤 삭제

구현한 선택 기능과 선택한 이유

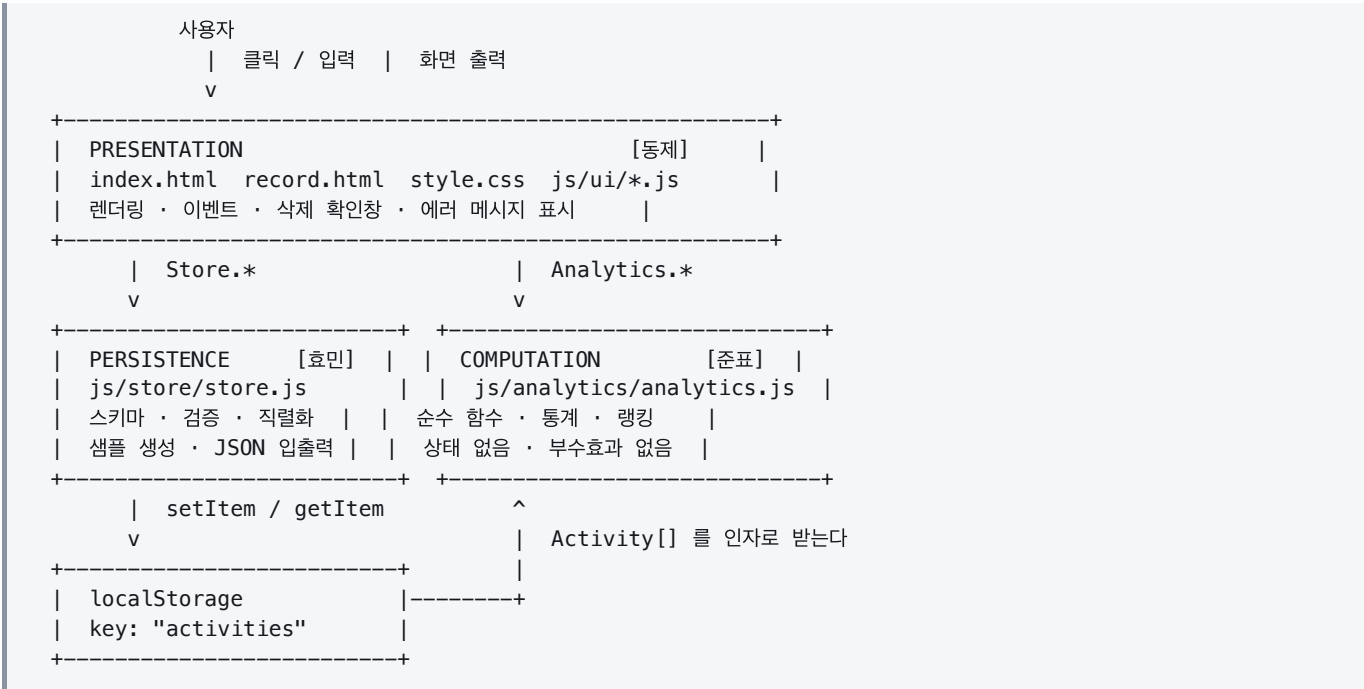
명세서 항목	구현	선택한 이유
월별 활동 횟수 통계 차트	<code>Analytics.monthlyCount()</code>	날짜가 이미 필수 필드라 추가 입력 없이 만들 수 있다
참여 인원 합계 및 평균 표시	<code>Analytics.summary()</code>	참여 인원이 필수 필드다. 집계 한 번으로 끝난다
데이터 JSON 내보내기 / 가져오기	<code>Store.exportJson()</code> <code>Store.importJson()</code>	서버가 금지되어 기기 간 공유가 불가능한데, 이 기능이 그 한계를 우회하는 유일한 수단이다
반응형 레이아웃	<code>style.css</code>	운동 직후 휴대폰으로 기록하는 상황이 주 사용 시나리오다

명세서 목록 밖의 자체 확장으로 체중 추이, 칼로리 수치, 감량 효과 랭킹을 추가했다. 차별화 요소이며 선택 기능을 대체하지 않는다.

2. 설계

데이터 아키텍처 — 3계층 단방향 의존

세 계층이 한 방향으로만 의존한다. 이 방향이 세 명이 서로를 기다리지 않고 동시에 작업할 수 있었던 근거다.



핵심은 **COMPUTATION 계층이 아래를 향하지 않는다**는 것이다. `analytics.js` 는 저장소를 모르고 배열을 인자로 받기만 한다. 그래서 `store.js` 가 아직 없던 시점에 배열을 손으로 만들어 통계 함수를 완성하고 검증할 수 있었다.

경계 규칙

계층	금지	이유
<code>ui.js</code>	<code>localStorage</code> 직접 접근	저장 형식이 바뀌면 화면까지 고쳐야 한다
<code>analytics.js</code>	<code>document</code> · <code>localStorage</code> · <code>Store</code> 참조	배열만 알면 어디서든 테스트된다
<code>store.js</code>	<code>alert</code> · DOM 조작	에러는 <code>errors</code> 객체로 돌려주고 표시 방식은 UI가 정한다

데이터 구조 — 저장 형식

서버·DB·빌드 도구가 금지되어 브라우저 `localStorage` 키 하나에 JSON 배열로 통째 저장한다. 제약이지만 이 앱에는 맞는 선택이기도 하다. 체중과 식단은 민감한 개인 정보고, 서버로 보내지 않으면 유출될 곳이 없다.

```
localStorage["activities"] = ' [{ "id": "a1", "title": "러닝 5km", ... }, { ... } ] '
```

	단일 키 (채택)	레코드마다 키 분리
읽기	전체 파싱 1회	키 목록 스캔 + 개별 파싱
쓰기	전체 직렬화 1회	해당 키만 갱신
정렬 · 집계	배열 그대로 사용	매번 모아서 재구성
코드	store.js 한 파일	키 관리 로직 추가

레코드가 수백 건 수준이라 전체 직렬화 비용이 문제되지 않는다. 단순함을 택했고 그 대가로 저장 책임이 한 파일에 모였다. 나중에 IndexedDB로 바꾸더라도 store.js 내부만 고치면 위 두 계층은 그대로다.

데이터 구조 — 필드 정의

명세서 고정 필드는 이름을 바꾸거나 지우지 않고 확장 필드만 덧붙였다.

필드	타입	필수	제약	쓰이는 곳
명세서 고정 필드 — 삭제 · 개명 금지				
id	string	자동	Store 발급, 중복 없음	삭제 대상 지정
title	string	O	공백 제거 후 1자 이상	목록 표시
date	string	O	YYYY-MM-DD, 미래 날짜 거부	월별 집계, 체중 추이, 랭킹
place	string	—	제한 없음	목록 표시
memberCount	number	O	1 이상 정수	인원 합계 · 평균
memo	string	—	제한 없음	목록 표시
createdAt	string	자동	ISO 문자열	최신순 정렬
확장 필드				
category	string	O	유산소 · 근력 · 스트레칭	종목별 집계, 칼로리 소모
durationMin	number	O	1 이상 정수	운동 시간 합계, 칼로리 소모
weightKg	number null	—	0 초과	체중 추이, 감량 효과 랭킹
kcalIn	number null	—	0 초과	칼로리 수치
exercise	string null	—	러닝 · 헬스 · 수영 · 자전거 · 등산	운동 랭킹 집계 키
foods	string[]	—	12종 목록에서 선택 (감량 7 · 증량 5)	음식 랭킹 집계 키

설계상 중요한 결정 세 가지

① 미입력은 null로 통일한다. undefined는 JSON.stringify에서 키째 사라져, 내보내기 → 가져오기 왕복에 필드 유무가 달라진다.

```
JSON.stringify({ a: undefined }) // '{}'          키가 사라짐
JSON.stringify({ a: null })      // '{"a":null}'   유지됨
```

실제로 26건 왕복 테스트에서 kcalIn: null 6건이 그대로 보존되는 것을 확인했다.

② 빈 문자열은 0 이 아니라 미입력이다. 체중 0kg 한 건이 섞이면 평균이 무너지고 추이 그래프가 바닥으로 꺾인다.

```
Number('') // 0 그냥 넘기면 체중 0kg 가 저장된다
Number.isInteger(parseInt('3.7')) // true 3 이 되어 정수 검사를 통과한다
```

③ exercise 와 foods 는 선택형 입력으로만 받는다. 둘 다 랭킹의 집계 키다. 자유 텍스트면 표본이 쪼개져 순위가 무너진다.

```
자유 텍스트 : "닭가슴살"(3건) "닭 가슴살"(2건) "닭가슴살 "(1건)
              -> 서로 다른 항목 3개. 전부 표본 3건 미만이라 순위에서 탈락
선택형       : "닭가슴살"(6건) -> 순위에 오른다
```

CRUD

모든 데이터 변경은 Store 를 통과한다. UI가 localStorage 를 직접 건드리는 경로는 없다.

Create — Store.add(input)

```
폼 입력
-> validate(input) ---- 실패 --> { ok:false, errors } --> UI 가 필드별 메시지 표시
    |                                     (localStorage 변화 없음)
    | 통과
    v
buildActivity(input)    id 발급 · createdAt 기록 · 타입 정규화 ('' -> null, 없는 foods -> [])
-> 기존 배열 + 새 항목
-> localStorage.setItem("activities", JSON.stringify(list))
-> { ok:true, activity }
```

id 와 createdAt 은 Store가 만든다. UI가 넘기지 않는다.

Read — Store.getAll()

```
localStorage.getItem("activities")
-> JSON.parse() ---- 깨진 값 --> [] (throw 하지 않는다)
-> createdAt 내림차순 정렬
-> Activity[]
    +--> 목록 화면                +--> Analytics.monthlyCount()
    +--> Analytics.summary()      +--> Analytics.kcalBalance()
    +--> Analytics.weightTrend()  +--> Analytics.topFoods() / worstFoods()
```

배열이 단일 진실 공급원이다. 화면마다 따로 저장하는 상태가 없어서 등록·삭제 후 getAll() 한 번만 다시 부르면 모든 화면이 같이 맞춰진다.

Update

활동 수정은 구현하지 않았다. 명세서 선택 기능이지만 우선순위에 밀렸다. 잘못 입력하면 삭제 후 재등록한다.

Delete — Store.remove(id)

```
확인창 (UI 담당)
-> 목록에서 해당 id 제외
-> 길이가 줄었는지 확인      없는 id 면 false
-> write(list) -> true
```

확인창은 UI가 띄우고 Store 는 화면을 모른 채 삭제만 한다.

파일 구조

```
hackathon1schoollover/
├── index.html           랜딩 화면
├── record.html          기록 · 통계 화면
├── style.css            스타일, 반응형 레이아웃
├── CLAUDE.md            세 명 공통 계약 (스키마 · 인터페이스 · 경계 규칙)
├── README.md
├── docs/
│   ├── 설계서.md       설계 결정 기록
│   └── AI_USAGE.md      Claude Code 위임 내역과 검증 방법
├── js/
│   ├── store/          store.js    [효민] + 담당 CLAUDE.md
│   ├── analytics/      analytics.js [준표] + 담당 CLAUDE.md
│   └── ui/              ui.js    landing.js [동제] + 담당 CLAUDE.md
```

file://에서는 ES 모듈이 CORS로 차단되므로 type="module" 을 쓰지 않고 일반 <script> 태그를 store → analytics → ui 순으로 로드해 전역 객체로 노출한다.

3. Claude Code 활용 과정

3-1. 작성한 CLAUDE.md

CLAUDE.md 를 루트 1개 + 담당 폴더별 3개로 나눴다. 루트에는 세 명 공통 계약(스키마·인터페이스·경계 규칙)을, 담당 폴더에는 그 사람 만 지킬 규칙을 뒀다.

넣은 규칙	왜 필요하다고 판단했는지
인터페이스 계약 시그니처를 먼저 고정	세 명이 동시에 작업하려면 상태 코드가 없어도 자기 몫을 끝낼 수 있어야 한다. 시그니처가 있으면 기다릴 필요가 없다
계층별 금지 사항 (경계 규칙 3줄)	파일 경계를 사람 경계와 일치시키면 병합 충돌이 나지 않는다. 실제로 5개 브랜치를 머지하는 동안 코드 충돌이 0건이었다
없는 값은 0으로 취급하지 않는다	0/0 = NaN 이고 NaN 이 화면에 찍히면 그대로 감점 스크린샷이 된다
서버 · DB · 빌드 도구 도입 금지 type="module" 금지	명세 제약이자 file:// 실행 전제다. 규칙으로 못 박지 않으면 AI가 익숙한 방식(모듈·번들러)으로 돌아간다
한 번에 100줄 넘는 코드를 통째 로 출력하지 않는다	큰 덩어리는 검증하지 않고 받아들이게 된다. 쪼개면 각 조각을 직접 확인하게 된다
커밋 본문에 검증 방법 한 줄	보고서를 쓸 때 기억에 의존하지 않아도 된다. 실제로 이 문서 대부분이 커밋 로그에서 나왔다

작업 도중 규칙을 수정한 내역

시각	무엇을	왜
14:05	역할 재정의 · 스키마 확장	명세가 서버를 금지하므로 "백엔드" 역할이 성립하지 않는다. 데이터 계층과 데이터 처리로 나눴다
14:25	담당별 문서 분리	루트 하나가 커지면서 자기와 무관한 규칙까지 읽어야 했다
14:35	다이어트 중심으로 방향 전환	주인공을 체중·칼로리로 옮겼다. 인터페이스 계약과 스키마는 건드리지 않아 진행 중이던 작업이 멈추 지 않았다
15:20	exercise · foods 2필 드 추가	이슈 #2 결정. 랭킹 기능을 살리기로 하면서 집계 키가 필요해졌다
15:40	weightTrend 중복 처리 명시	담당 문서가 "정하고 적어라"로 열어둔 항목을 실제로 닫았다 (그날 마지막 기록 사용)

3-2. 구현 전 계획

계획 화면 캡처 첨부 위치. 13:40-14:00 사이 설계 확정 화면 (스키마 · 파일 소유자 · 구현 순서).

실제 작업이 계획과 달라진 부분

- **ML 기능의 정의가 바뀌었다.** 처음에는 "날씨 기반 운동 추천"으로 잡았으나, 실제로 원한 것은 "내 몸에 효과가 있었던 음식·운동 Top3"였다. 날씨는 그 Top3를 날씨에 배치하는 보조 축으로 내려갔다 (3-4 참조)
- **날씨 기능이 계획에서 빠졌다.** 구현까지 마쳤으나 이슈 #2에서 B-2로 연기하고 필수 기능을 먼저 끝내기로 했다. 판단 근거는 `file://` 에서 외부 `fetch` 가 막힐 수 있다는 점이었다
- **활동 수정 기능을 잘라냈다.** 폼 재사용을 위해 등록 로직을 리팩터해야 해서 비용이 컸다
- **계획에 없던 이슈 트래커를 썼다.** 계약 충돌이 생겼을 때 구두로 정리되지 않아 GitHub 이슈에 선택지와 결정 시한을 적고 세 명이 코멘트로 결정했다

3-3. 핵심 프롬프트 3건

① 요청이 명세 제약과 충돌하는 것을 먼저 잡아낸 사례 효민

무엇을 요청했나	"운동동아리를 만들려고해. 헬스 운동을 기록하고 동아리원의 오늘 운동 부위 등을 볼 수 있는. 알람도 가게끔. 앱이 나올까 웹이 나올까?"
어떤 결과를 받았나	앱/웹 답변보다 먼저 두 기능이 명세 제약에 걸린다 는 지적이 돌아왔다. (1) "동아리원의 오늘 운동 부위 보기"는 localStorage가 브라우저 하나에 갇혀 있어 기기 간 공유가 불가능하고 서버가 필요한데 명세가 금지한다. (2) 브라우저 알림은 페이지가 열려 있는 동안만 뜨고, <code>file://</code> 로 열면 권한 요청 자체가 차단된다
만족스러웠는지	만족스러웠다. 요청대로 만들었다면 시연 중에 동작하지 않는 기능 두 개 를 꺼내고 갔을 것이다

② 통계 엔진을 저장소 없이 먼저 완성한 사례 준표

무엇을 요청했나	"m작업할 건데 (...) 한 달동안 이 사람이 어떤 운동을 하였고, 어떤 식단을 하였으며, 얼마나 몸의 변화가 일어났는지 분석하고 (...) 동아리는 주제는 다이어트 동아리야. 우리는 웹으로 구현할거야."
어떤 결과를 받았나	제약 충돌 3건을 먼저 지적받았다. (1) 명세가 서버를 금지하므로 "백엔드" 역할이 성립하지 않는다 (2) 오늘 만든 앱의 localStorage는 0건이라 분석을 시연하려면 샘플 생성 기능이 필요하다 (3) 스키마가 템플릿에 고정되어 있으므로 지우지 말고 확장해야 한다. 그 다음 ML 방식으로 TensorFlow.js 대신 순수 JS 최소자승 회귀 + 피어슨 상관을 권고받았다 — 외부 의존이 0이라 데모 중에 터지지 않고 보고서에 수식을 적을 수 있다는 이유였다
만족스러웠는지	만족스러웠다. 특히 <code>analytics.js</code> 가 <code>Store</code> 를 모르게 설계한 덕분에 <code>store.js</code> 가 없던 시점에 배열을 손으로 만들어 통계 함수를 전부 완성하고 검증 할 수 있었다

③ 컨셉이 명세 검증 요건과 충돌한 것을 잡은 사례 효민

무엇을 요청했나	"이번 주 다이어트 계획 을 입력한다"를 원래 컨셉으로 잡고 진행을 요청했다
어떤 결과를 받았나	명세서 입력값 검증 요건에 " 날짜는 미래 날짜 입력 불가 "가 있다는 지적. 계획은 본질적으로 미래 날짜라 컨셉과 검증 요건이 정면으로 충돌한다
만족스러웠는지	만족스러웠다. 컨셉을 "계획 입력"에서 " 기록 후 분석 "으로 돌렸다. 검증 요건을 지키면서 오히려 "내 데이터로 검증된 문장을 만든다"는 지금의 정체성이 여기서 나왔다

3-4. 프롬프트 개선 사례

처음 요청한 내용

날씨에 따라 그럼 사람들이 어떤 스포츠를 선호하는지도 찾아서 자세히 어떤 운동을 어떤 시간대에 하면 좋은지도 같이 분석해서 덧붙이면 좋을 것 같아.

결과가 기대와 달랐던 지점

요청대로 **날씨를 중심에 둔 기능**이 나왔다. Open-Meteo 예보를 받아 체감온도·강수확률로 시간대를 점수화하고 최적 2시간 구간을 고르는 엔진이었다. 동작은 했지만 방향이 어긋나 있었다.

- 주인공이 **날씨**가 되어 있었다. 정작 알고 싶은 건 "내 몸에 뭐가 효과 있었나"였다
- 운동 분류가 카테고리 3종(유산소·근력·스트레칭) 단위라 너무 거칠었다. "러닝이 효과적"이 아니라 "유산소가 효과적"까지만에 못 나온다
- **식단 분석이 아예 없었다**. 스키마에 `kcalIn` (칼로리 총량)만 있고 음식 종류가 없었다
- 추천이 오늘 하루치뿐이라 "다음 날짜들"이라는 요구를 만족하지 못했다

요청을 어떻게 고쳤는지

지금 날씨로 예측 `ml`이 주요 기능이 아니야. 내 몸이 어떤 음식과 어떤 운동에서 가장 체중 감량이 많이 되었고, 그 중에서도 top3를 매겨 다음 날짜들에 맞춰서 날씨 기준으로 잡고 예측을 해주는 거야.

무엇이 본체이고 무엇이 보조인지를 명시했다. 처음 요청은 "날씨에 따라 ~도 덧붙이면"이라는 추가 요청 형태여서, 무엇을 중심에 둘지가 빠져 있었다.

고친 뒤 무엇이 달라졌는지

	처음	고친 뒤
본체	날씨 기반 시간대 추천	개인 효과 랭킹 (날씨는 배치 필터로 강등)
분석 단위	카테고리 3종	개별 운동 5종 + 개별 음식 12종
식단	없음	<code>foods</code> 필드 추가, 음식별 감량 기여도 산출
산출 범위	오늘 2시간 구간	앞으로 7일 계획 + 예상 감량 누적

계산 모델도 상관분석에서 **귀속(attribution)** 방식으로 바뀌었다.

$$\Delta w(d) = \frac{(\text{다음 기록일 체중} - \text{당일 체중})}{\text{경과일수}}$$

하루당으로 정규화
경과일수 3일 초과 구간은 인과를 특정할 수 없어 버린다
그날 한 운동과 먹은 음식 각각에 $\Delta w(d)$ 를 귀속 -> 항목별 평균으로 정렬
표본 3건 미만은 순위에서 제외, 양수 항목은 '회피 항목'으로 분리

검증용 데이터에 러닝 -0.15kg/일 , 헬스 -0.08kg/일 , 요가 $+0.05\text{kg/일}$ 을 심어두고 돌렸더니 **심어둔 값을 소수점 셋째 자리까지 정확히 복원**했다. 두 운동일에 걸친 닭가슴살이 두 값의 평균(-0.115)으로 나온 것까지 맞았다.

얻은 교훈은 "무엇을 추가할지"보다 "무엇이 중심인지"를 먼저 말해야 한다는 것이다. 추가 요청 형태로 말하면 시는 직전 맥락을 중심으로 삼는다.

3-5. 오류 해결 사례

발생한 오류

16:26, 마감 14분 전. 세 사람의 브랜치를 합치고 화면을 처음 열었더니 **필수 3기능이 하나도 렌더되지 않았다**.

활동 목록 : "저장 모듈(`store.js`)을 기다리는 중입니다."
기록 추가 : 폼이 비어 있음
통계 : `hidden`

콘솔 에러는 `favicon.ico` 404 하나뿐이었다. **에러 없이 조용히 실패하는 상태**였다.

원인 파악 과정

스크립트 로드 순서(`store` → `analytics` → `ui`)와 `type="module"` 미사용을 먼저 확인했다. 둘 다 정상이었다. 다음으로 모듈이 실제로 정의됐는지 브라우저 콘솔에서 확인했다.

typeof Store	-> "object"	정의되어 있다
typeof Analytics	-> "object"	정의되어 있다
window.Store	-> undefined	<-- 여기
window.Analytics	-> undefined	

ui.js 가 모듈 존재를 확인하는 방식이 window 를 보고 있었다.

```
function ready(name) {  
  return typeof window[name] !== 'undefined' && window[name];  
}
```

그런데 store.js 와 analytics.js 는 둘 다 최상위 const 로 선언한다. 최상위 const · let 은 전역 렉시컬 스코프에 들어갈 뿐 window 의 프로퍼티가 되지 않는다. var 나 함수 선언이었다면 붙었을 것이다. 그래서 ready() 가 항상 false 를 반환하고 render() 가 첫 줄에서 빠져나갔다.

해결 방법

각 모듈 끝에 한 줄씩 추가했다. CLAUDE.md 가 "전역 객체로 노출한다"고 한 계약을 실제로 만족시키는 방향이다.

```
// js/store/store.js  
window.Store = Store;  
// js/analytics/analytics.js  
window.Analytics = Analytics;
```

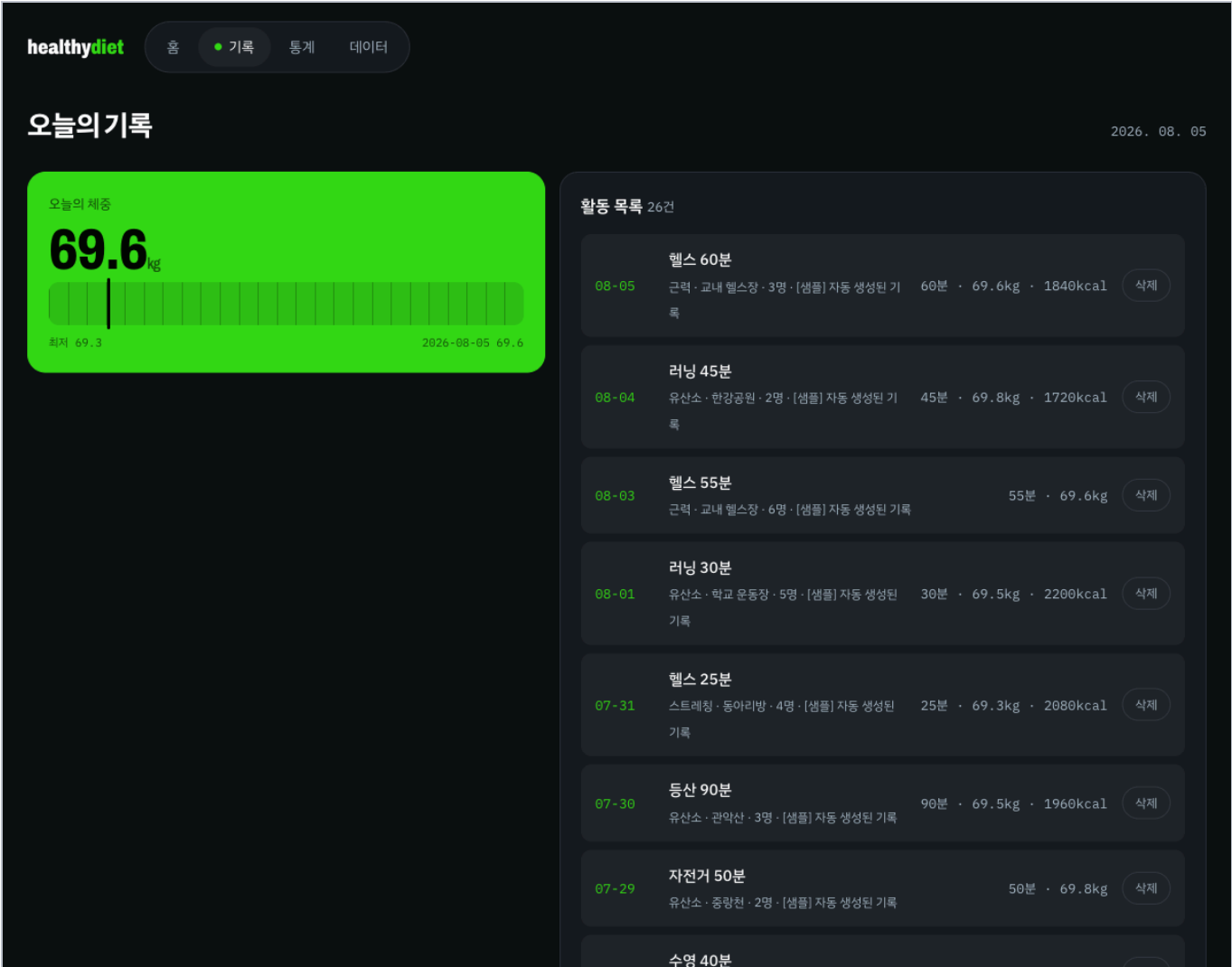
ui.js 의 ready() 를 고치는 선택지도 있었지만, 모듈 두 개에 한 줄씩 넣는 쪽이 변경이 작고 계약 문구와도 맞아서 그쪽을 택했다.

이 과정에서 배운 점

- "에러가 없다"가 "정상이다"는 아니다. 콘솔이 깨끗한데도 화면이 비어 있었다. 존재 확인이 조용히 실패하면 아무 흔적도 남기지 않는다
- 세 파일 모두 자기 기준으로는 정상이었다. store.js 도 analytics.js 도 "전역으로 노출한다"는 계약을 지켰다고 믿었고, ui.js 도 "전역을 확인한다"를 구현했다. 전역이라는 단어의 해석이 달랐을 뿐이다. 계약은 문장이 아니라 실행되는 코드로 확인해야 한다
- 통합 확인을 마지막에 물면 남는 시간이 없다. 이 버그를 14분 전에 발견했다. 모듈 단위 테스트는 셋 다 통과하고 있었고, 세 파일이 한 페이지에서 만나야만 드러나는 종류였다

같은 성격의 사례가 하나 더 있었다. 통합 테스트에서 감량 랭킹이 정확히 반대로 나왔다 (감량 1위 피자, 회피 대상 단백질셰이크). 원인은 analytics.js 가 "기록 이후의 체중 변화를 그 기록에 귀속"시키는데 seedSampleData() 는 오늘의 선택을 오늘 체중에 이미 반영해 저장했기 때문이다. 한 칸씩 밀려 귀속되어 다음 날 항목의 효과가 오늘 항목에 붙었다. 두 모듈 다 자기 기준으로는 정상이었고, 모델의 시점 정의가 어긋난 것이 통합에서만 드러났다. 증상이 "틀린 값"이 아니라 "정확히 반대"였던 것이 방향 문제라는 단서였다.

4. 실행 결과





화면 2 — 통계. 활동 수 26 · 참여 인원 합계 103 · 평균 4.0 · 운동 시간 1290분 · 평균 50분, 월별 활동 횟수, 종목별, 감량 효과 Top3(운동·음식)와 피해야 할 항목. 증감은 색이 아니라 방향 삼각형(▼ 감량 / ▲ 증량)으로 표시하고 표본 수를 함께 적는다.

테스트한 항목과 결과

병합된 `main` 을 브라우저에서 직접 열어 확인한 내역이다.

구분	항목	결과
필수 기능	활동 등록	저장 성공 (26건 → 27건)
	활동 목록 조회	최신순 렌더 정상, 빈 목록 안내 문구 정상
	활동 삭제	확인창 표시 후 삭제 (27건 → 26건)
입력값 검증	활동명 빈값	거부
	미래 날짜	거부 — <code>input max=2026-08-05</code> 로 1차 방어, <code>store.js</code> 로 2차 방어. 오늘 날짜는 정상 저장
	참여 인원 0 · -1 · 2.5 · "" · "abc"	전부 거부
	빈 문자열 체중 · 칼로리	0 이 아니라 <code>null</code> 로 저장
선택 기능	참여 인원 합계 · 평균	103 / 4.0
	월별 활동 횟수	2026-07 22회, 2026-08 4회
	JSON 내보내기 → 가져오기 왕복	26건 → 26건, 필드 집합 동일, <code>kcalIn: null</code> 6건 보존
	깨진 JSON · 배열 아닌 입력	<code>throw</code> 없이 <code>{ ok:false }</code> , 검증 실패 항목은 <code>skipped</code> 로 집계
분석 엔진	감량 효과 Top3	운동: 자전거 · 등산 · 수영 / 음식: 계란 · 현미밥 · 닭가슴살
	회피 항목	치킨 · 탄산음료 · 피자
화면 오류	<code>NaN</code> · <code>undefined</code> 노출	없음
	콘솔 에러	<code>favicon.ico</code> 404 하나뿐

샘플 데이터는 최근 30일 중 26건을 생성하며 연속 공백 최대 2일, `weightKg` 는 전 기록에 존재, `kcalIn` 은 일부 비워 "없는 값 건너뛰기" 경로가 실제로 테스트되도록 구성했다.

5. 한계와 개선 방향

시간 내 구현하지 못한 것

마감 후 보완. 제출 시점에는 감량 효과 Top3가 화면에 표시되지 않았다. 엔진은 완성·검증됐지만 `ui.js` 가 `Analytics.topFoods()` 를 호출하지 않아 차별화 기능이 사용자에게 닿지 않는 상태였다. 총평 이후 렌더 블록을 추가해 현재는 통계 화면에서 확인할 수 있다 (PR #11).

항목	상태와 이유
날씨 기반 7일 계획 (<code>recommend.js</code>)	구현·검증 완료했으나 이슈 #2에서 B-2로 연기해 <code>BJP</code> 브랜치에 남았다. 외부 API 의존이라 필수 기능보다 뒤로 미룬 판단이었다
활동 수정	명세서 선택 기능이지만 폼 재사용을 위해 등록 로직을 리팩터해야 해서 잘라냈다
활동명 · 장소 키워드 검색	가장 저렴한 선택 기능이었으나 통합 디버깅에 시간을 쓰면서 밀렸다

현재 코드의 문제점이나 불안한 부분

- 감량 효과는 인과가 아니라 상관이다. 같은 날 여러 항목을 하면 효과가 나뉘어 귀속된다. 표본이 적으면 우연이 순위에 섞인다. 그래서 표본 수를 함께 표시하고 3건 미만은 순위에서 뺐지만, "닭가슴살이 살을 빼준다"가 아니라 "닭가슴살을 먹은 날의 평균 변화가 이렇다"로 읽어야 한다
- 칼로리 소모는 거친 추정치다. 분 × 카테고리별 계수 로만 계산하고 체중과 개인차를 반영하지 않는다. 정밀도를 흉내내지 않으려고 의도적으로 단순하게 뒀지만 절대값을 신뢰할 수는 없다

- 전역 객체 3개에 의존한다. Store · Analytics · Recommend 가 전역 이름을 쓰므로 이름이 겹치면 조용히 덮인다. 실제로 이번에 window 노출 누락으로 화면 전체가 비는 사고가 났다
- 브라우저 하나에만 저장된다. 기기·브라우저 간 공유가 안 되고, 브라우저 데이터를 지우면 기록이 사라진다
- 자동화 테스트가 없다. 모든 검증을 콘솔과 화면 확인으로 했다. 회귀를 막을 장치가 없다

시간이 더 있었다면 무엇을 했을지

1. 통합 확인을 15:00에 한 번 강제한다. 이번 두 건의 오류(랭킹 반전, window 노출 누락)는 모듈이 만나야만 보이는 종류였고 둘 다 마감 직전에 발견됐다. 빈 화면이라도 세 파일을 한 번 붙여보는 시점을 일정에 넣었어야 했다
2. recommend.js 를 합류시킨다. 실패 함수와 오프라인 경로가 이미 들어가 있어 위험은 제거된 상태다
3. 서버를 붙여 동아리원 간 공유를 만든다. 지금은 수동 JSON 교환이 유일한 공유 수단이다

6. 역할 분담과 소감

팀원별 담당 업무

팀원	담당	주요 산출물
이동제	프론트엔드	index.html, record.html, style.css, js/ui/ui.js, landing.js. 루트 CLAUDE.md 와 담당별 문서 3개 초안. 외부 CDN(AOS) 실패 시 폴백 처리
박호민	데이터 계층	js/store/store.js (계약 7종, 검증 규칙, 샘플 생성, JSON 입출력), docs/설계서.md, 열려 있던 결정 5건 확정, 이슈 #2 최종 결정
봉준표	데이터 처리	js/analytics/analytics.js (계약 5종 + 감량 효과 랭킹 6종), recommend.js, README.md, 통합 검증 및 마감 직전 블로커 진단

AI를 활용한 개발에 대해 느낀 점

이동제 — 규칙 파일을 먼저 쓰는 것이 생각보다 큰 차이를 만들었다. CLAUDE.md 에 " type="module" 금지", "서버 도입 금지"를 못 박아두지 않았다면 AI는 익숙한 방식으로 돌아갔을 것이고, 그때는 file:// 에서 백지 화면을 보고 나서야 알았을 것이다.

박호민 — AI가 가장 크게 기여한 지점은 코드를 빨리 짜준 것이 아니라 요청이 명세와 충돌한다고 먼저 말해준 것이었다. "동아리원 운동 부위 공유", "알람", "이번 주 계획 입력" 세 가지 모두 그대로 만들었다면 시연 중에 동작하지 않는 기능을 꺼안고 갔을 것이다. 반대로 아쉬운 것은, 모듈이 각자 맞아도 모델이 어긋나면 통합에서 터진다는 점이다. 랭킹이 정반대로 나온 문제는 두 파일이 만나야만 보였다.

봉준표 — 프롬프트에서 무엇이 본체인지 말하는 것이 결정적이었다. "날씨에 따라 ~도 덧붙이면"이라고 요청했을 때는 날씨가 주인공인 기능이 나왔고, "날씨 예측이 주요 기능이 아니야, 내 몸에 뭐가 효과 있었는지가 본체야"라고 고치자 완전히 다른 설계가 나왔다. AI는 직전 맥락을 중심으로 삼기 때문에, 추가 요청 형태로 말하면 우선순위가 뒤바뀐다. 그리고 "에러가 없다"가 "정상이다"는 아니라는 것을 마감 14분 전에 배웠다. 콘솔이 깨끗한데도 화면이 비어 있었고, 세 파일 모두 자기 기준으로 계약을 지키고 있었다.